

.NET Core Tutorial

A Complete Guide

15 chapters compiled into one PDF

Snehal Kadiya

+91 9974031480

snehaliteng@gmail.com

<https://snehaliteng.github.io/>

Copyright & Disclaimer

© 2026 Snehal Kadiya. All rights reserved.

This tutorial is intended for personal learning and educational purposes only. No part of this document may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the author.

Please do not print this document unnecessarily. Think of the environment — save paper and trees. Share the digital link instead:

<https://snehaliteng.github.io/tutorials/dotnetcore/>

Getting Started

What is .NET Core?

.NET Core is a cross-platform, open-source, modular framework for building modern applications. Unlike the traditional .NET Framework, .NET Core runs on Windows, Linux, and macOS. It was designed from the ground up to be lightweight, fast, and suitable for cloud and microservices architectures.

Cross-Platform

One of the most significant advantages is its cross-platform nature. You can develop and deploy on any supported operating system without recompiling. This makes it ideal for containerized environments like Docker and hybrid cloud deployments.

Open-Source

The entire .NET Core runtime, libraries, and tooling are open-source under the MIT license. The source code is hosted on GitHub, and the community actively contributes features, bug fixes, and improvements.

Modular Design

.NET Core is modular through NuGet packages. You include only the packages you need, reducing the footprint of your application. This is a departure from the monolithic .NET Framework where the entire base class library was bundled.

.NET vs .NET Framework vs Mono

Understanding the ecosystem is essential. The classic .NET Framework is Windows-only and ships with the OS. Mono is an open-source implementation

that enabled .NET on other platforms before .NET Core existed. .NET Core (now simply .NET 5/6/7/8+) unifies all three into a single cross-platform offering.

As of .NET 5, "Core" was dropped from the name. The latest versions are simply called .NET 6, .NET 7, .NET 8, etc. This tutorial covers the modern .NET platform.

Prerequisites

Before you begin, install the .NET SDK from dotnet.microsoft.com/download. The SDK includes the runtime, the `dotnet` CLI, and the compiler.

Verify Installation

```
dotnet --version
```

This should print something like `8.0.100`.

First Steps: Hello World

Create a new console application and run it:

```
dotnet new console -n HelloWorld  
cd HelloWorld  
dotnet run
```

The `dotnet new console` command scaffolds a minimal project with a single `Program.cs` file that prints `"Hello, World!"` to the console.

Modify the Program

Open `Program.cs` in any text editor and change the output:

```
Console.WriteLine("Hello from .NET Core!");
```

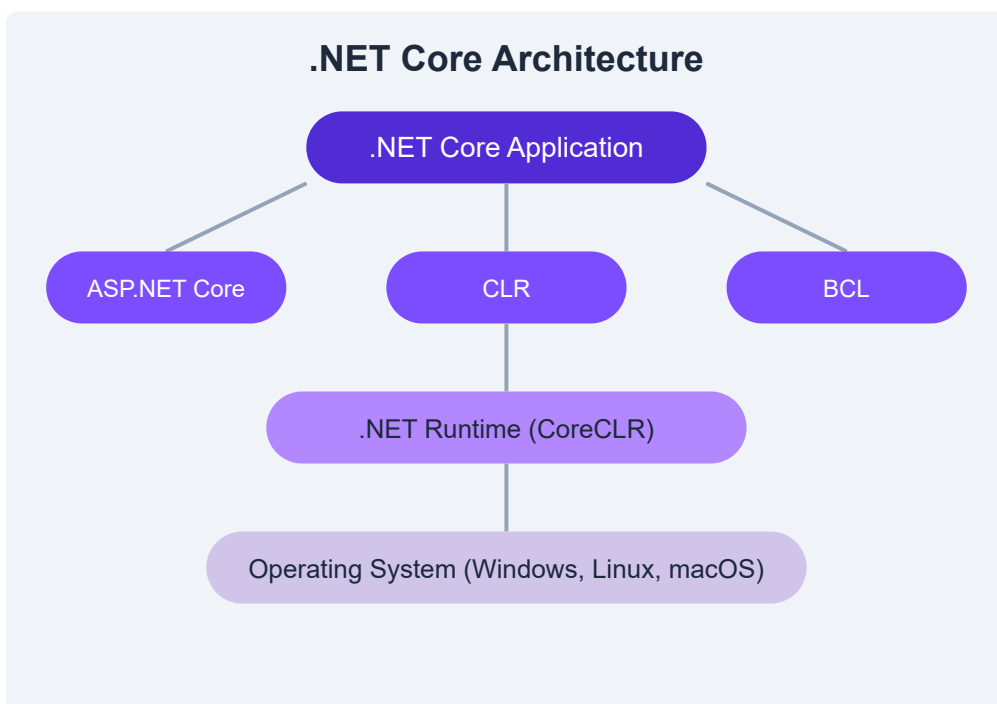
Run again with `dotnet run` to see your change.

Project Structure

A basic .NET project consists of:

- **Program.cs** — the entry point of the application
- **.csproj** — the project file that defines dependencies, target framework, and build settings
- **bin/** and **obj/** — build output and intermediate directories (generated, not checked into source control)

Understanding the Build Process



The image above illustrates the high-level architecture of .NET Core: the runtime (CoreCLR), the base class libraries (CoreFX), and the SDK/tooling layer work together to build and execute your application.

.NET Core applications are self-contained or framework-dependent. Self-contained apps include the runtime; framework-dependent apps require the runtime to be installed on the target machine.

Exercise: Create a new console app named `MyFirstApp`, modify `Program.cs` to print your name, and run it. Experiment with `dotnet build` and observe the files created in `bin/`.

Development Environment

The .NET SDK

The .NET SDK (Software Development Kit) includes everything you need to build and run .NET applications: the runtime, the compiler, the `dotnet` CLI, and the MSBuild engine. Install it once and you are ready to develop on any platform.

SDK vs Runtime

The **runtime** is required to run applications. The **SDK** is required to build them. If you are deploying a framework-dependent app, only the runtime needs to be on the target machine. For development, always install the SDK.

The `dotnet` CLI

The `dotnet` command-line interface is the primary tool for all .NET development tasks. It provides a consistent experience across operating systems.

Common CLI Commands

```
dotnet new <template>    # Create a new project
dotnet restore            # Restore NuGet dependencies
dotnet build              # Build the project
dotnet run                # Build and run
dotnet test               # Run unit tests
dotnet publish            # Publish for deployment
dotnet pack               # Create a NuGet package
```

Visual Studio

Visual Studio is a full-featured IDE available in three editions:

- **Community** — Free for individual developers, open-source contributors, and small teams
- **Professional** — Paid edition with advanced tooling for professional developers
- **Enterprise** — Premium edition with architecture tools, load testing, and advanced debugging

Visual Studio provides IntelliSense, debugging, profiling, integrated testing, and project templates. It is the most productive choice for Windows developers.

VS Code with C# Dev Kit

For a lightweight, cross-platform editor, Visual Studio Code with the C# Dev Kit extension is excellent. It provides:

- IntelliSense and code navigation
- Built-in terminal for `dotnet` commands
- Debugging support
- Git integration
- Extensions for testing, Docker, and more

Project Templates

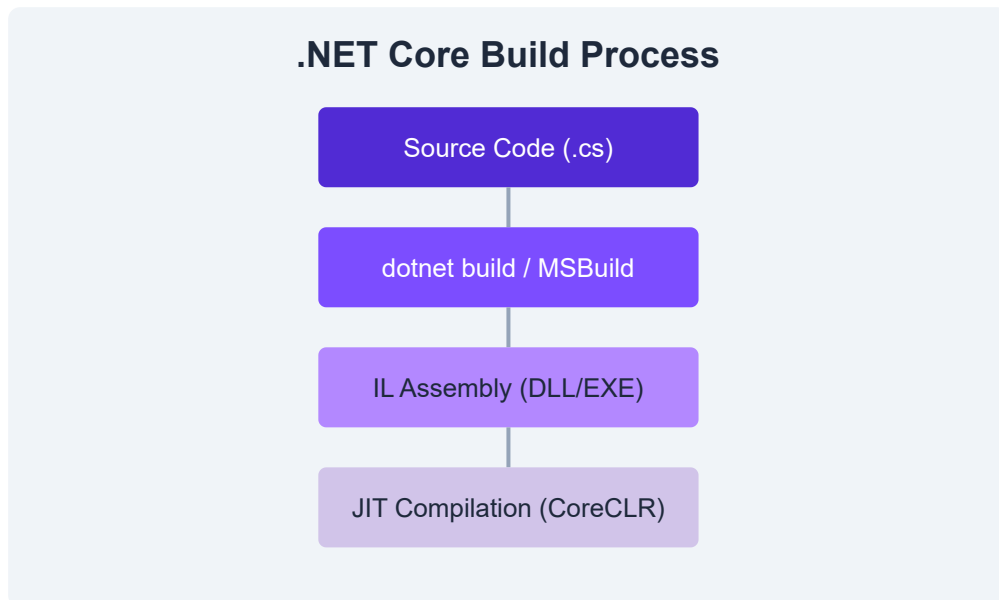
The `dotnet new` command supports many built-in templates:

```
dotnet new console      # Console application
dotnet new web          # ASP.NET Core empty web app
dotnet new webapi       # ASP.NET Core Web API
dotnet new mvc          # ASP.NET Core MVC
dotnet new classlib     # Class library
dotnet new xunit        # xUnit test project
```

List all available templates with `dotnet new list`.

You can also install third-party templates using `dotnet new install <package-id>`, which enables sharing project scaffolding across teams.

Choosing the Right IDE



Your choice depends on your workflow. Visual Studio is best for large enterprise solutions with many projects. VS Code is ideal for cross-platform development, microservices, and containerized apps. JetBrains Rider is another popular cross-platform IDE with excellent .NET support.

Build Process Overview

The diagram above shows how source code, project files, and dependencies flow through the build pipeline. The `dotnet build` command compiles your C# code into Intermediate Language (IL) that the runtime can execute.

Exercise: Create a new Web API project (`dotnet new webapi -n MyApi`), open it in your chosen editor, and run it. Browse to the Swagger endpoint to see the auto-generated API documentation.

Numerics & Core APIs

Numeric Types

.NET provides a rich set of numeric types covering integers, floating-point, and arbitrary-precision arithmetic.

Integral Types

```
int a = 42;           // 32-bit signed integer
long b = 42L;         // 64-bit signed integer
short c = 42;         // 16-bit signed integer
byte d = 42;          // 8-bit unsigned integer
uint e = 42U;         // 32-bit unsigned integer
ulong f = 42UL;       // 64-bit unsigned integer
```

Floating-Point Types

```
float f = 3.14f;      // 32-bit, ~7 digits precision
double d = 3.14;      // 64-bit, ~15 digits precision
decimal m = 3.14m;    // 128-bit, 28-29 digits precision, exact decimal
```

Use `decimal` for financial and monetary calculations where exact representation is crucial. Use `double` for scientific computing. Use `float` for graphics and GPU computations.

BigInteger

For arbitrarily large integers, use `System.Numerics.BigInteger`:

```
using System.Numerics;

BigInteger huge = BigInteger.Parse("123456789012345678901234567890");
BigInteger result = huge * huge;
Console.WriteLine(result);
```

Math and MathF

The `Math` class provides static methods for trigonometric, logarithmic, and algebraic operations on `double`. `MathF` is the `float` equivalent:

```
double sqrt = Math.Sqrt(144);           // 12.0
double power = Math.Pow(2, 10);          // 1024.0
double pi = Math.PI;                     // 3.14159...
float sin = MathF.Sin(MathF.PI / 2);     // 1.0f
```

String Operations

Strings are immutable in .NET. Common operations include concatenation, interpolation, and parsing:

```
string name = "Alice";
string greeting = $"Hello, {name}!";
string upper = greeting.ToUpper();
int length = greeting.Length;
```

DateTime and TimeSpan

```
DateTime now = DateTime.Now;
DateTime utc = DateTime.UtcNow;
DateTime birthday = new DateTime(1990, 6, 15);
TimeSpan age = now - birthday;
int daysOld = (int)age.TotalDays;
```

Collections

.NET provides several collection types for different scenarios:

```
List<string> names = new List<string> { "Alice", "Bob", "Charlie" };
names.Add("Diana");

Dictionary<string, int> scores = new()
{
    ["Alice"] = 95,
    ["Bob"] = 87
};

HashSet<int> unique = new() { 1, 2, 3, 3 }; // contains {1, 2, 3}
Queue<string> queue = new();               // FIFO
Stack<string> stack = new();               // LIFO
```

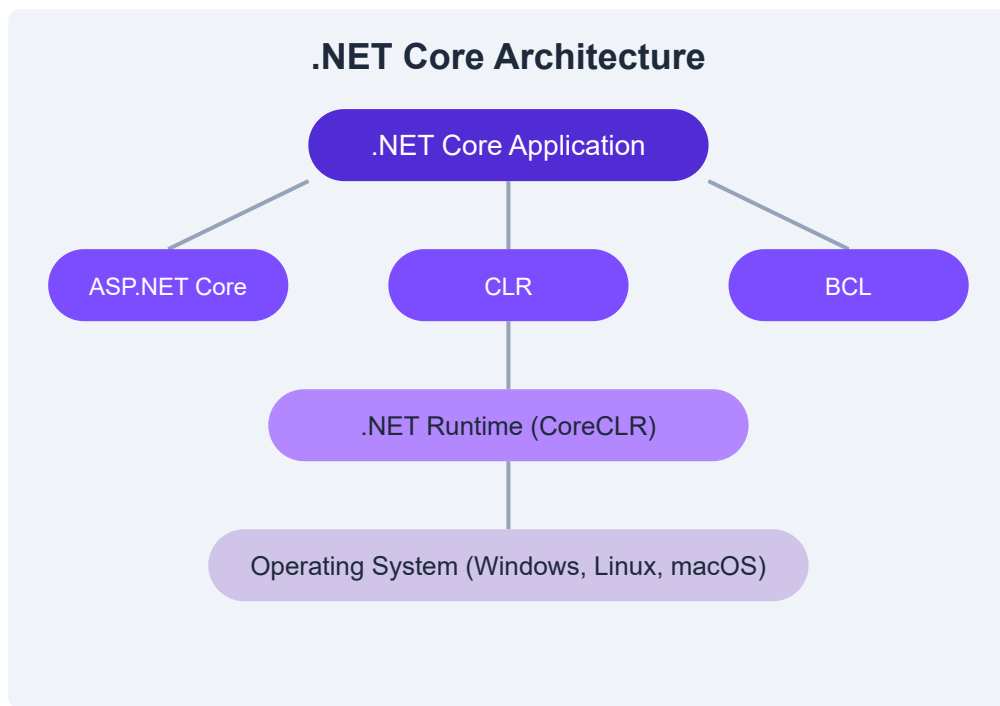
LINQ Queries

Language Integrated Query (LINQ) allows you to query collections with a SQL-like syntax or method chains:

```
int[] numbers = { 5, 2, 8, 3, 1, 9, 4 };

var evenNumbers = numbers.Where(n => n % 2 == 0);
var sorted = numbers.OrderBy(n => n);
var grouped = numbers.GroupBy(n => n % 2 == 0 ? "even" : "odd");
var sum = numbers.Aggregate((a, b) => a + b);
```

LINQ queries are lazily evaluated. Use `.ToList()` or `.ToArray()` to materialize results immediately.



Exercise: Write a console app that reads a list of integers from the user, filters only prime numbers using LINQ, computes their sum and average, and prints the result.

Memory Management

Stack vs Heap Allocation

The stack stores value types and reference pointers with a fixed lifetime tied to the method scope. The heap stores reference type instances and dynamically sized data. Stack allocations are faster because they use a simple pointer adjustment; heap allocations require garbage collection overhead.

```
int value = 42;           // Stored on the stack
object obj = new object(); // Reference on the stack, object on the heap
```

Garbage Collection

The .NET garbage collector (GC) manages heap memory automatically. It runs in generations:

- **Gen 0** — Short-lived objects collected most frequently
- **Gen 1** — Objects that survived a Gen 0 collection
- **Gen 2** — Long-lived objects collected infrequently
- **Large Object Heap (LOH)** — Objects $\geq 85,000$ bytes are stored on a separate heap collected as Gen 2

GC Modes

The GC can run in workstation mode (default) or server mode (optimized for multi-processor throughput). Server GC uses separate heaps per core to reduce contention.

Forcing a Collection

```
GC.Collect();           // Forces a blocking collection
GC.WaitForPendingFinalizers(); // Waits for finalizers to complete
```

Avoid calling `GC.Collect()` in production code. The runtime optimizes collection timing based on memory pressure and allocation patterns.

IDisposable and the `using` Statement

Unmanaged resources (file handles, database connections, network sockets) must be released explicitly. Implement `IDisposable` to clean them up:

```
using var file = new StreamReader("data.txt");
string content = file.ReadToEnd();
// file.Close() is called automatically at the end of the scope
```

The `using` statement ensures `Dispose()` is called even if an exception is thrown.

Finalizers

A finalizer (destructor) is called by the GC before reclaiming memory. Finalizers should only release unmanaged resources:

```
class ResourceHolder
{
    ~ResourceHolder() => Cleanup();
    private void Cleanup() { /* release resources */ }
}
```

Weak References

Weak references let you hold a reference to an object without preventing its collection. Useful for caches:

```
WeakReference<MyClass> weak = new(new MyClass());  
if (weak.TryGetTarget(out var obj))  
{  
    Console.WriteLine("Object still alive");  
}
```

Memory Leaks Prevention

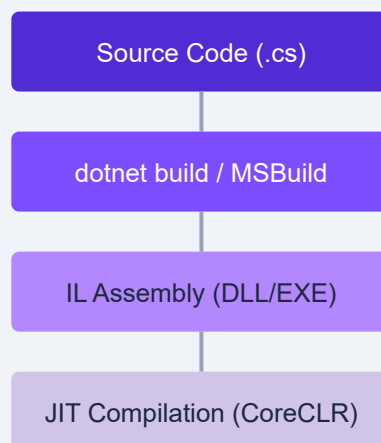
Common leak sources include: event handlers that are never unsubscribed, static collections holding object references, and large objects that pin memory. Use memory profiling tools like dotMemory or PerfView to diagnose leaks.

Span<T> and Memory<T>

These stack-allocatable types enable high-performance memory access without allocations:

```
Span<int> span = stackalloc int[] { 1, 2, 3, 4, 5 };  
span[0] = 10; // Modifies in-place, no heap allocation
```

.NET Core Build Process



Exercise: Create a console app that allocates a large number of objects (e.g., 1 million strings), force a GC collection, and measure memory usage before and after using `GC.GetTotalMemory()` .

Code Execution

Common Language Runtime (CoreCLR)

CoreCLR is the runtime that executes .NET applications. It provides garbage collection, security, exception handling, and type safety. When you run a .NET application, CoreCLR loads and executes the compiled Intermediate Language (IL) code.

Intermediate Language (IL)

The C# compiler (`csc`) compiles source code into IL, a CPU-independent instruction set. IL is packaged in assemblies (`.dll` or `.exe`) along with metadata and resources.

```
// C# source
int Add(int a, int b) => a + b;

// Resulting IL (simplified)
// ldarg.0
// ldarg.1
// add
// ret
```

Just-In-Time Compilation (JIT)

At runtime, CoreCLR's JIT compiler translates IL into native machine code for the current architecture. JIT compiles methods on demand, caching the native code for subsequent calls. This enables optimizations tailored to the specific CPU and operating system.

JIT compilation incurs a startup cost because methods are compiled the first time they are called. This can be mitigated with Tiered Compilation, which compiles quickly first and recompiles hot methods with full optimization later.

Ahead-Of-Time Compilation (NativeAOT)

NativeAOT compiles the entire application into a single native executable with no dependency on CoreCLR at runtime. Benefits include:

- Instant startup (no JIT phase)
- Smaller deployment footprint
- No runtime installation required
- Harder to reverse-engineer

```
dotnet publish -c Release -r win-x64 --self-contained  
# Add <PublishAot>true</PublishAot> in your .csproj to enable NativeAOT
```

NativeAOT has limitations: no dynamic code generation, limited reflection, and no runtime loading of arbitrary assemblies. It is ideal for tools, CLI apps, and container images where fast startup matters.

Entry Points

Historically, the entry point was the `Main` method. Starting with C# 9, top-level statements simplify this:

```
// Traditional Main
class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Hello");
    }
}

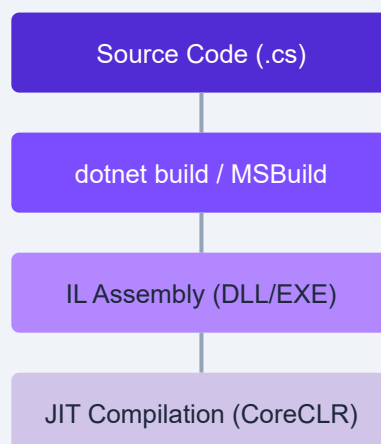
// Top-level statements (C# 9+)
Console.WriteLine("Hello");
```

Assembly Loading and Versions

Assemblies are loaded by the Assembly Loader using probing, codebase, or the Global Assembly Cache (GAC). Assembly versioning uses four-part numbers (major.minor.build.revision) and binding redirects control which version loads at runtime.

```
Assembly asm = Assembly.Load("MyLibrary, Version=1.0.0.0");
Type type = asm.GetType("MyLibrary.MyClass");
var instance = Activator.CreateInstance(type);
```

.NET Core Build Process



Exercise: Create a console app, publish it with both standard JIT and NativeAOT configurations. Compare the startup time using `Stopwatch` and examine the published output sizes.

Modularity

Projects and Solutions

.NET organizes code into **projects** (single `.csproj` file) and **solutions** (`.sln` file) that group multiple projects together. Solutions are ideal for multi-project applications like a web API plus a class library plus a test project.

```
dotnet new sln -n MyApp
dotnet new webapi -n MyApp.Api
dotnet new classlib -n MyApp.Core
dotnet new xunit -n MyApp.Tests
dotnet sln add MyApp.Api MyApp.Core MyApp.Tests
```

Solution Structure

Once added, the solution file tracks each project's path and configuration. Building the solution builds all projects in dependency order:

```
dotnet build MyApp.sln
dotnet test MyApp.sln
```

NuGet Packages

NuGet is the package manager for .NET. Packages contain compiled libraries, tools, and metadata. The NuGet gallery at nuget.org hosts hundreds of thousands of packages.

Adding a NuGet Package

```
dotnet add package Newtonsoft.Json
dotnet add package Microsoft.EntityFrameworkCore.SqlServer --version 8.0.0
```

Creating and Publishing a NuGet Package

```
dotnet pack -c Release
dotnet nuget push bin/Release/MyPackage.1.0.0.nupkg --api-key YOUR_KEY --sour
```

Dependency Injection

The `Microsoft.Extensions.DependencyInjection` library provides a first-class DI container. It is the foundation of ASP.NET Core and can be used in any .NET application.

Service Lifetimes

```
// Transient – new instance every time
services.AddTransient<IMyService, MyService>();

// Scoped – same instance per request/scope (common in web apps)
services.AddScoped<IMyService, MyService>();

// Singleton – single instance for the application lifetime
services.AddSingleton<IMyService, MyService>();
```

Consuming Dependencies

```
public class HomeController
{
    private readonly IMyService _service;

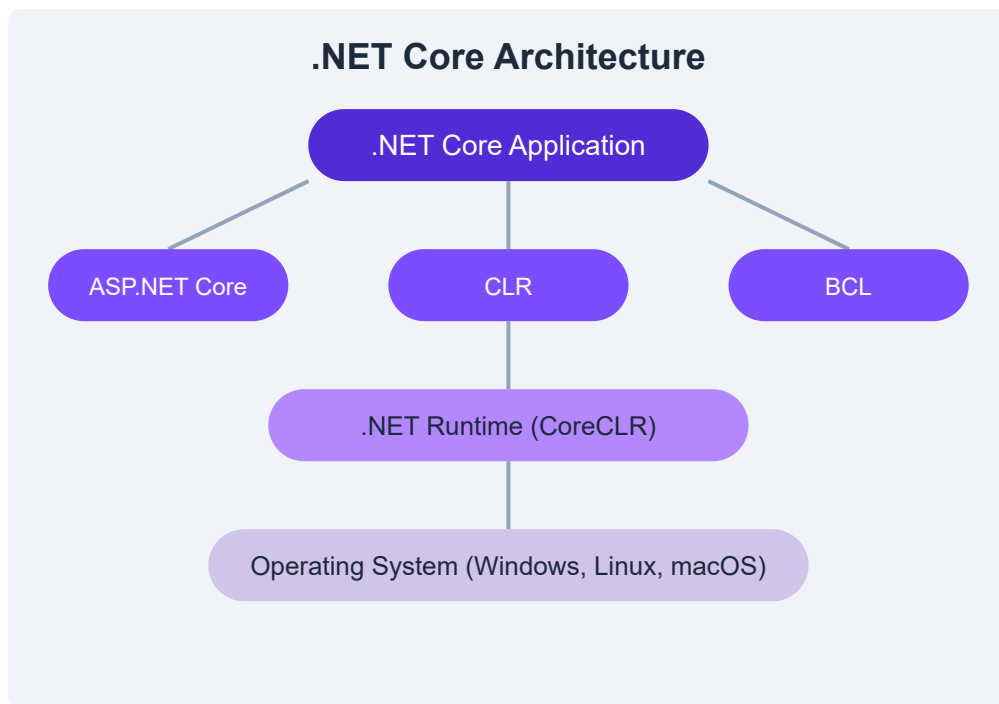
    public HomeController(IMyService service)
    {
        _service = service;
    }
}
```

Configuration System

The `Microsoft.Extensions.Configuration` package provides a hierarchical key-value configuration system supporting JSON files, environment variables, command-line args, and more.

```
// appsettings.json
{
    "Logging": { "LogLevel": { "Default": "Information" } },
    "ConnectionStrings": {
        "Default": "Server=.;Database=MyDb;Trusted_Connection=True"
    }
}

// Reading configuration
var connectionString = configuration.GetConnectionString("Default");
var logLevel = configuration["Logging:LogLevel:Default"];
```



Exercise: Create a solution with a class library and a console app. Add a NuGet reference from the console app to the class library. Register services using DI and read a connection string from `appsettings.json`.

Project & Build System

The SDK-Style `.csproj` Format

Modern .NET uses a simplified, SDK-style `.csproj` format that is much cleaner than the legacy format. It is XML-based but requires far fewer lines.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <OutputType>Exe</OutputType>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Newtonsoft.Json" Version="13.0.3" />
  </ItemGroup>
</Project>
```

The `Sdk` attribute tells MSBuild which set of build targets and tasks to import. Common SDKs are `Microsoft.NET.Sdk` (console, classlib, test) and `Microsoft.NET.Sdk.Web` (ASP.NET Core).

MSBuild Concepts

MSBuild is the build engine underlying all .NET builds. It processes XML-based project files and executes targets (build steps) composed of tasks (individual operations).

Properties

```
<PropertyGroup>
  <TargetFramework>net8.0</TargetFramework>
  <LangVersion>12</LangVersion>
  <ImplicitUsings>enable</ImplicitUsings>
  <Nullable>enable</Nullable>
</PropertyGroup>
```

Items

```
<ItemGroup>
  <Compile Remove="OldCode\**" />
  <EmbeddedResource Include="Resources\**\*" />
  <PackageReference Include="Microsoft.AspNetCore.App" />
</ItemGroup>
```

Targets and Tasks

```
<Target Name="CustomStep" AfterTargets="Build">
  <Message Text="Build completed!" Importance="high" />
  <Copy SourceFiles="output.dll" DestinationFolder="..\deploy\" />
</Target>
```

Metapackages

Metapackages reference a group of related NuGet packages without adding any code of their own:

- **Microsoft.NETCore.App** — The base runtime and framework libraries
- **Microsoft.AspNetCore.App** — All ASP.NET Core packages (MVC, Razor, SignalR, etc.)
- **Microsoft.NETCore.App.Ref** — Reference assemblies for targeting a specific runtime version

For web projects, use the `Microsoft.NET.Sdk.Web` SDK which automatically references `Microsoft.AspNetCore.App`. You do not need to add it as a `PackageReference`.

Restoring and Building

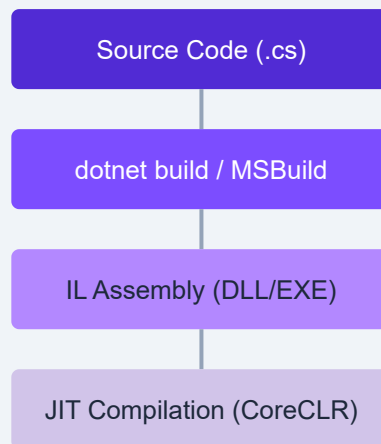
```
dotnet restore    # Downloads missing packages and resolves dependencies
dotnet build      # Compiles the project incrementally
dotnet build --no-restore # Skips restore for faster incremental builds
```

`dotnet restore` is implicit in `dotnet build`, `dotnet run`, and `dotnet test` unless you pass `--no-restore`.

Common MSBuild Properties

Property	Description
<code>TargetFramework</code>	Target framework moniker (e.g., <code>net8.0</code> , <code>netstandard2.0</code>)
<code>OutputType</code>	Exe, Library, WinExe, Module
<code>LangVersion</code>	C# language version (e.g., <code>12</code> , <code>latest</code>)
<code>Nullable</code>	Enable nullable reference types (<code>enable</code> / <code>disable</code> / <code>annotations</code> / <code>warnings</code>)
<code>ImplicitUsings</code>	Auto-import common namespaces (<code>enable</code> / <code>disable</code>)
<code>PublishAot</code>	Enable NativeAOT compilation (<code>true</code> / <code>false</code>)

.NET Core Build Process



Exercise: Create a class library project and inspect the auto-generated `.csproj`. Add a custom MSBuild target that copies the output DLL to a `deploy` folder after each build. Verify it runs with `dotnet build`.

Libraries & Extensions

Creating Class Libraries

Class libraries encapsulate reusable logic that can be shared across multiple applications. Create one with:

```
dotnet new classlib -n MyLibrary
cd MyLibrary
dotnet build
```

This generates a `.csproj` targeting `net8.0` (or your current SDK version) with `<OutputType>Library</OutputType>`.

.NET Standard vs Target Framework Moniker (TFM)

.NET Standard is a formal specification of APIs that all .NET implementations must implement. It allows libraries to work across .NET Framework, .NET Core, and Xamarin.

TFM	Target
<code>netstandard2.0</code>	Broadest compatibility (all .NET implementations)
<code>netstandard2.1</code>	Latest standard (not supported by .NET Framework 4.x)
<code>net8.0</code>	.NET 8 (modern, cross-platform)
<code>net6.0</code>	.NET 6 (LTS, widely deployed)

For new libraries targeting only modern .NET, use a specific TFM like `net8.0`. If you need compatibility with .NET Framework or Xamarin, target `netstandard2.0`.

NuGet Package Creation

A NuGet package bundles a compiled library (`.dll`) with metadata, dependencies, and optional content files.

Using `.nuspec`

```
<?xml version="1.0"?>
<package>
  <metadata>
    <id>MyLibrary</id>
    <version>1.0.0</version>
    <authors>YourName</authors>
    <description>A useful library</description>
  </metadata>
</package>
```

Pack and Push

```
dotnet pack -c Release
dotnet nuget push bin\Release\MyLibrary.1.0.0.nupkg --api-key YOUR_KEY --source
```

You can also publish to a local feed, Azure Artifacts, or GitHub Packages by changing the `--source` URL.

UWP Apps and Windows Runtime (WinRT) Components

Universal Windows Platform (UWP) apps can consume .NET Standard libraries. WinRT components (created in C++/CX or C#) expose APIs that can be consumed by any UWP language (C#, JavaScript, C++).

Extension SDKs

Extension SDKs provide platform-specific APIs for Windows development (e.g., Windows Mobile SDK, Xbox Live SDK). They are referenced in the project file

and resolved by the build system based on the target platform.

Portable Class Libraries (PCL)

PCLs were the predecessor to .NET Standard. They allowed targeting specific combinations of platforms (e.g., iOS + Android + Windows Phone) by defining a "profile" of available APIs.

PCLs are deprecated in favor of .NET Standard. If you maintain a PCL, migrate it to `netstandard2.0` to gain access to more APIs and broader compatibility.

Sharing Libraries Across Projects

You can reference a library project directly in a solution or via a NuGet package. Direct project references are simpler during development:

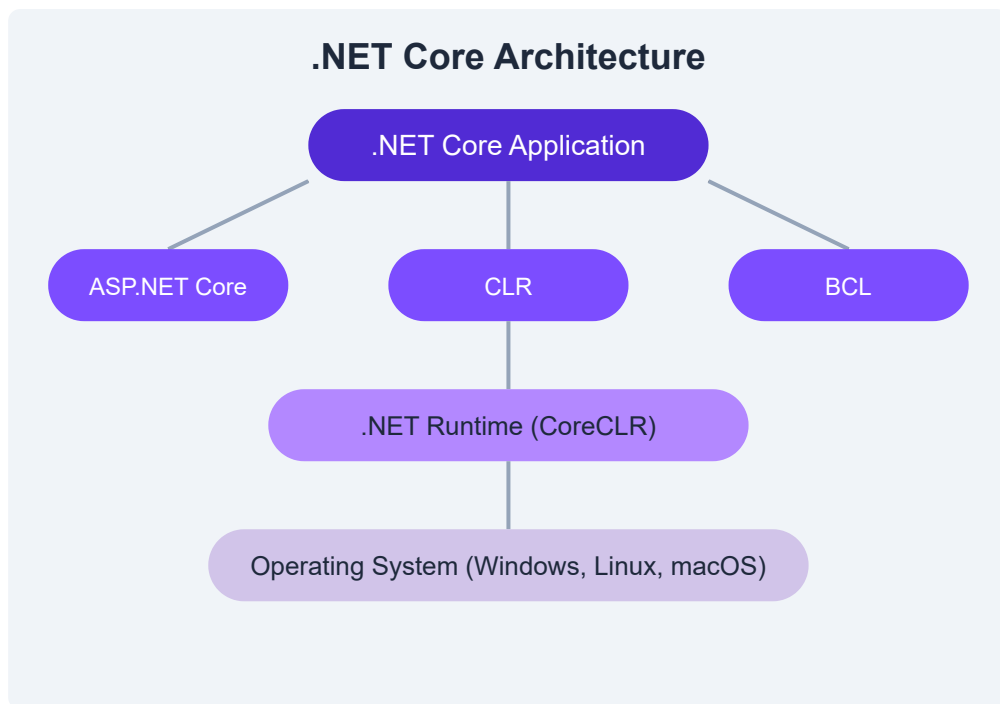
```
dotnet add ConsoleApp.csproj reference ..\MyLibrary\MyLibrary.csproj
```

Xamarin.Forms

Xamarin.Forms allows sharing UI code across iOS, Android, and Windows using a single .NET class library. Shared logic lives in a .NET Standard library, while platform-specific renderers and services live in individual platform projects.

PCL Troubleshooting

Common issues include missing API members due to profile limitations, mismatched platform versions, and assembly binding failures. Use the `PlatformNotSupportedException` guard to handle missing APIs gracefully.



Exercise: Create a class library with a utility method (e.g., a string extension method), package it as a NuGet package using `dotnet pack`, and consume it from a console application. Publish the package to a local feed.

Testing

Testing is a critical part of any professional .NET Core application. This chapter covers the most popular testing frameworks, how to write unit tests, mocking dependencies, and adopting a test-driven development workflow.

Creating a Test Project

The .NET SDK includes built-in templates for xUnit, NUnit, and MSTest projects. The most popular choice is xUnit.

```
dotnet new xunit -n MyApp.Tests  
cd MyApp.Tests  
dotnet add reference ../MyApp/MyApp.csproj
```

This creates a test project with xUnit pre-configured. Add a reference to the project you want to test.

Writing Tests with xUnit

xUnit uses the `[Fact]` attribute for parameterless test methods and `[Theory]` for data-driven tests.

```

public class CalculatorTests
{
    [Fact]
    public void Add_ReturnsSum()
    {
        var calc = new Calculator();
        var result = calc.Add(2, 3);
        Assert.Equal(5, result);
    }

    [Theory]
    [InlineData(1, 2, 3)]
    [InlineData(-1, -1, -2)]
    [InlineData(0, 0, 0)]
    public void Add_WithMultipleInputs_ReturnsExpected(int a, int b, int expected)
    {
        var calc = new Calculator();
        var result = calc.Add(a, b);
        Assert.Equal(expected, result);
    }
}

```

Assert Class

The `Assert` class provides a rich set of static methods: `Equal`, `NotEqual`, `True`, `False`, `Null`, `NotNull`, `Throws`, `InRange`, and collection assertions like `Contains` and `All`.

Live-Dependency Testing with Moq

Moq is the most widely used mocking library for .NET. It lets you create fake implementations of interfaces.

```

public interface IUserRepository
{
    User GetById(int id);
}

[Fact]
public void GetUser_ReturnsUser_WhenExists()
{
    var mockRepo = new Mock<IUserRepository>();
    mockRepo.Setup(r => r.GetById(1))
        .Returns(new User { Id = 1, Name = "Alice" });

    var service = new UserService(mockRepo.Object);
    var user = service.GetUser(1);

    Assert.Equal("Alice", user.Name);
    mockRepo.Verify(r => r.GetById(1), Times.Once);
}

```

Moq is available via `dotnet add package Moq`. Consider **NSubstitute** or **FakeItEasy** as lightweight alternatives.

NUnit and MSTest Alternatives

NUnit uses `[Test]` and `[TestCase]`. It supports setup/teardown via `[SetUp]` and `[TearDown]`. **MSTest** uses `[TestMethod]` and `[DataTestMethod]` with `[DataRow]`. All three frameworks integrate seamlessly with `dotnet test`.

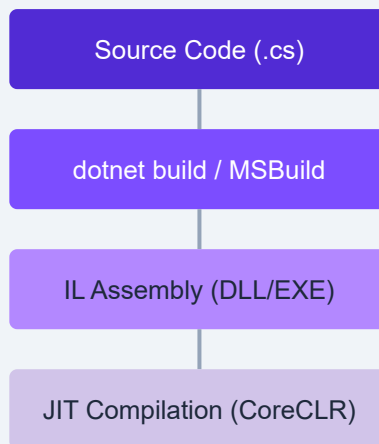
Test-Driven Development (TDD) Workflow

TDD follows a simple red-green-refactor cycle:

1. **Red:** Write a failing test for the desired behavior.
2. **Green:** Write the minimum code to make the test pass.
3. **Refactor:** Improve the code while keeping tests green.

This process produces well-tested, clean, and maintainable code.

.NET Core Build Process



Running Tests

Use the CLI or Visual Studio's Test Explorer:

```
dotnet test # Run all tests
dotnet test --filter "Category=Unit" # Filter by trait
dotnet test --logger "trx;LogFileName=results.trx" # Export results
```

Test Explorer in VS/VS Code provides a graphical view of passed, failed, and skipped tests with inline diagnostics.

Code Coverage

Use `dotnet-coverage` or `Coverlet` to measure code coverage:

```
dotnet add package coverlet.msbuild
dotnet test /p:CollectCoverage=true /p:CoverletOutputFormat=opencover
```

Aim for at least 70-80% coverage on business logic. Focus on testing behavior, not implementation details.

Integration Testing

For ASP.NET Core apps, use `Microsoft.AspNetCore.Mvc.Testing` with a `WebApplicationFactory` to spin up an in-memory test server:

```
public class ApiTests : IClassFixture<WebApplicationFactory<Program>>
{
    private readonly HttpClient _client;

    public ApiTests(WebApplicationFactory<Program> factory)
    {
        _client = factory.CreateClient();
    }

    [Fact]
    public async Task Get_Endpoint_ReturnsOk()
    {
        var response = await _client.GetAsync("/api/values");
        response.EnsureSuccessStatusCode();
    }
}
```

Exercise: Create an xUnit test project for a simple `StringUtils` class that has methods `Reverse(string)`, `IsPalindrome(string)`, and `CountWords(string)`. Write at least 5 `[Fact]` and 2 `[Theory]` tests, then run `dotnet test` to verify all pass.

SDK & CLI

The .NET SDK and its command-line interface (CLI) are the foundation for creating, building, testing, and deploying .NET applications. Mastering the CLI unlocks efficient cross-platform development workflows.

dotnet new — Project Templates

The `dotnet new` command scaffolds projects from built-in or custom templates. List available templates with `dotnet new list`.

```
dotnet new console -n MyConsole
dotnet new webapi -n MyApi
dotnet new mvc -n MyMvcApp
dotnet new classlib -n MyLibrary
dotnet new xunit -n MyTests
dotnet new sln -n MySolution
```

Use `--output` to specify a target directory and `--framework` to target a specific TFM like `net8.0`.

Custom Templates

You can create your own templates by adding a `.template.config/template.json` folder to a project and installing it via `dotnet new install <path>`.

dotnet build — Compiling Code

`dotnet build` compiles the project and its dependencies. Internally it invokes MSBuild with the designated target.

```
dotnet build                                # Debug build
dotnet build -c Release                      # Release build
dotnet build --no-restore                   # Skip restore if already done
dotnet build --verbosity normal             # Control log level
```

The output is written to `bin/Debug/<tfm>/` or `bin/Release/<tfm>/`.

dotnet test — Running Tests

`dotnet test` discovers and runs tests in one or more test projects.

```
dotnet test
dotnet test --filter "FullyQualifiedName~Integration"
dotnet test --logger "console;verbosity=detailed"
dotnet test --settings test.runsettings
```

It supports all major frameworks (xUnit, NUnit, MSTest) and can produce coverage reports via Coverlet.

dotnet run — Executing Applications

`dotnet run` compiles and runs a project in one step. Useful during development.

```
dotnet run                                # Run with default profile
dotnet run -- --port 5000                 # Pass arguments to app
dotnet run -c Release                     # Run release build
dotnet run --project src/MyApp            # Target a specific project
```

For production, use `dotnet publish` to create a self-contained deployment rather than `dotnet run`.

dotnet publish — Deploying Applications

`dotnet publish` compiles the application and copies all necessary files to a deployment folder.

```
dotnet publish -c Release -o ./publish
dotnet publish --self-contained true --runtime win-x64
dotnet publish -p:PublishSingleFile=true
dotnet publish -p:PublishReadyToRun=true
```

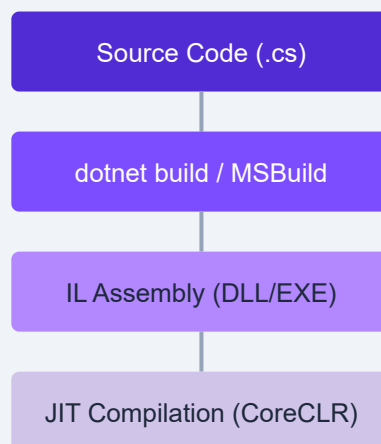
dotnet pack — Creating NuGet Packages

```
dotnet pack -c Release
dotnet pack -o ./nupkgs
dotnet pack -p:Version=2.0.0
```

dotnet nuget — Package Management

```
dotnet nuget push MyPackage.1.0.0.nupkg --api-key ... --source https://api.nuget.org/v3/index.json
dotnet nuget delete MyPackage 1.0.0 --non-interactive
dotnet nuget locals all --clear
dotnet nuget list source
```

.NET Core Build Process



global.json and Roll-Forward Policy

A `global.json` file in the project root pins the SDK version and configures roll-forward behavior:

```
{
  "sdk": {
    "version": "8.0.100",
    "rollForward": "latestMajor",
    "allowPrerelease": false
  }
}
```

Roll-forward policies include `patch`, `feature`, `minor`, `major`, `latestPatch`, `latestFeature`, `latestMinor`, `latestMajor`, and `disable`.

Use `dotnet --version` to check the current SDK version and `dotnet --list-sdks` to see all installed SDKs.

Exercise: Create a new solution `CliDemo` containing a console app and a class library. Add NuGet package `Newtonsoft.Json` to the library, build in Release mode, run the app, and publish a self-contained deployment for your OS. Commit the `global.json` that pins the SDK to your current version.

Build & Deployment

This chapter explores the build system in depth, CI/CD pipeline integration, Docker containerization, and the various deployment models available for .NET Core applications.

MSBuild Deep Dive

MSBuild is the build engine behind .NET Core. Every `.csproj` file is an MSBuild project file. You can extend the build pipeline with custom targets.

Custom Targets

```
<Target Name="PostBuild" AfterTargets="Build">
  <Copy SourceFiles="$(OutputPath)$(AssemblyName).dll"
        DestinationFolder="$(SolutionDir)dist/" />
</Target>
```

BeforeBuild / AfterBuild

Override the built-in `BeforeBuild` and `AfterBuild` targets to inject pre- or post-processing logic:

```
<Target Name="BeforeBuild">
  <Message Importance="high" Text="Building $(ProjectName)..." />
</Target>
```

MSBuild Properties and Items

Common properties: `OutputType`, `TargetFramework`, `AssemblyName`, `RootNamespace`. Items like `Compile`, `Content`, `EmbeddedResource` define what gets built.

CI/CD Pipelines

Modern teams automate builds and deployments using continuous integration and delivery pipelines.

GitHub Actions

```
name: .NET Core Build
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup .NET
        uses: actions/setup-dotnet@v4
        with:
          dotnet-version: 8.0.x
      - name: Restore
        run: dotnet restore
      - name: Build
        run: dotnet build --no-restore -c Release
      - name: Test
        run: dotnet test --no-build -c Release
      - name: Publish
        run: dotnet publish -c Release -o artifacts
```

Azure DevOps Pipelines

```
trigger:
- main
pool: ubuntu-latest
steps:
- task: DotNetCoreCLI@2
  inputs:
    command: 'test'
    arguments: '--configuration Release'
```

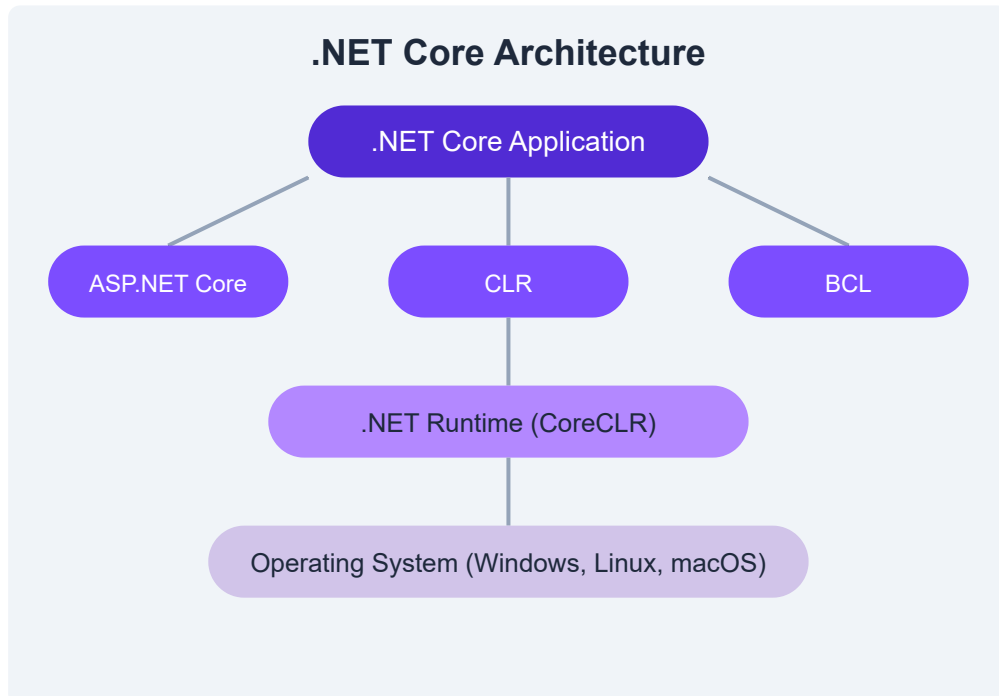
Docker Containers

Containerizing .NET apps is straightforward with multi-stage Dockerfiles:

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY . .
RUN dotnet publish -c Release -o /app

FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS runtime
WORKDIR /app
COPY --from=build /app .
EXPOSE 8080
ENTRYPOINT ["dotnet", "MyApp.dll"]
```

Use the `aspnet` image for ASP.NET Core apps and the smaller `runtime` image for console apps. Consider Alpine-based images for smaller footprints.



Deployment Models

Framework-dependent deployment (FDD) requires the .NET runtime on the target machine. It produces smaller deployments.

Self-contained deployment (SCD) bundles the runtime and framework libraries together. Larger but entirely independent.

```
dotnet publish --self-contained true --runtime win-x64
dotnet publish --self-contained false           # Default
```

Single-File Publishing

Packages the entire application into a single executable:

```
dotnet publish -r win-x64 --self-contained true -p:PublishSingleFile=true -p:
```

ReadyToRun (R2R)

R2R compiles IL to native code at publish time, reducing JIT overhead at startup:

```
dotnet publish -c Release -r win-x64 --self-contained true -p:PublishReadyToR
```

R2R images are larger but start significantly faster — ideal for containerized and serverless workloads.

Trimming

Publish trimming removes unused assemblies to reduce deployment size:

```
dotnet publish -p:PublishTrimmed=true -p:TrimMode=Link
```

Exercise: Take any .NET Core web API project. Create a GitHub Actions workflow that builds, runs tests, publishes a self-contained single-file executable for `linux-x64`, and builds a Docker image. Verify the workflow passes on push.

Migrations

Moving from the classic .NET Framework to .NET Core (modern .NET) requires careful planning. This chapter covers migration strategies, tooling, breaking changes, and best practices for large codebases.

Upgrading from .NET Framework

Modern .NET (5, 6, 7, 8+) is the successor to both .NET Core and .NET Framework. The recommended path is:

1. Run the **.NET Portability Analyzer** to assess compatibility.
2. Convert `packages.config` to `PackageReference`.
3. Retarget the project to a compatible `.NET Standard` version (2.0 is widely compatible).
4. Upgrade to the latest .NET SDK and recompile.

project.json to csproj Conversion

Early .NET Core used `project.json`; the SDK later returned to the `.csproj` format. If you still have `project.json`:

```
dotnet migrate                # Automated migration tool
# Or manually create a .csproj:
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
  </PropertyGroup>
</Project>
```

The `dotnet migrate` command was deprecated after .NET Core 3.0. For newer projects, create a new SDK-style project and port your code manually.

API Compatibility Analyzer

The **API Compatibility Analyzer** (`Microsoft.DotNet.ApiCompat`) helps identify APIs that are not available on your target platform. It runs as an MSBuild task:

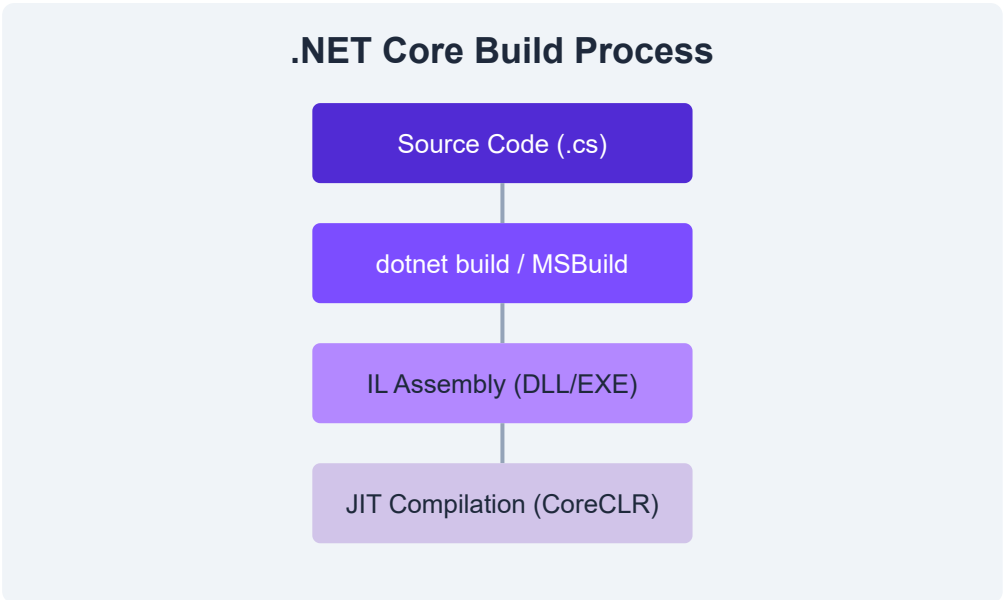
```
<PackageReference Include="Microsoft.DotNet.ApiCompat" Version="8.0.0" />
```

Run `dotnet build` and review any compatibility warnings.

Breaking Changes

Several technologies from .NET Framework are not available in modern .NET:

- **System.Drawing:** Limited to Windows. Use `SkiaSharp` or `ImageSharp` for cross-platform image processing.
- **AppDomain:** Code isolation via AppDomains is replaced by `AssemblyLoadContext`.
- **Remoting:** .NET Remoting is removed. Use gRPC, SignalR, or message queues instead.
- **BinaryFormatter:** Unsafe and removed. Use `System.Text.Json` or `Protobuf`.
- **Enterprise Services (COM+):** Not supported. Use dependency injection instead.



Porting Analyzer Tool

The `dotnet-apiport` tool (now integrated into Visual Studio) scans your assemblies and generates a detailed report of API usage and compatibility.

```
dotnet tool install -g dotnet-apiport
dotnet apiport analyze -f MyApp.exe -t net8.0 -o report.html
```

.NET Standard Compatibility

.NET Standard 2.0 is the bridge between .NET Framework and modern .NET. Libraries targeting .NET Standard 2.0 can be consumed by both ecosystems.

.NET Standard	Supported by
2.0	.NET Framework 4.6.1+, .NET Core 2.0+, Unity 2018.1+
2.1	.NET Core 3.0+, .NET 5+ (not supported by .NET Framework)

If you need to support .NET Framework consumers, target `netstandard2.0`. For new libraries, target `net8.0` (or later) to access the latest APIs.

Tips for Large Codebases

- Migrate incrementally — convert one project at a time using a strangler fig pattern.
- Use `#if NET` preprocessor directives to maintain dual-target builds during transition.
- Lock your NuGet dependency versions and test each upgrade step.
- Automate the build after each project migration to catch regressions early.

Dual-Targeting Strategy

You can target both .NET Framework and modern .NET during migration:

```
<TargetFrameworks>net472;net8.0</TargetFrameworks>
```

Use `#if NET472` and `#if NET8_0` in source code to isolate platform-specific code.

Exercise: Take a small .NET Framework console app (targeting 4.7.2) that uses `System.Drawing` to resize images and `AppDomain` for plugin loading. Port it to .NET 8, replacing `System.Drawing` with `SkiaSharp` and `AppDomain` with `AssemblyLoadContext`. Run the portability analyzer before and after.

Advanced Topics

This chapter explores advanced .NET Core features for performance optimization, extensibility, interop with native code, diagnostics, reflection, and source generators.

Performance Optimization

Modern .NET provides several high-performance APIs that minimize allocations and improve throughput.

Span<T> and Memory<T>

`Span<T>` is a stack-allocated, ref-like type that provides safe access to contiguous memory without allocations. `Memory<T>` is its heap-compatible counterpart.

```
Span<byte> buffer = stackalloc byte[256];
ReadOnlySpan<char> text = "Hello".AsSpan();
int vowels = text.Count(SpanExtensions.IsVowel);

// Slicing without allocation
Span<byte> slice = buffer.Slice(10, 20);
```

ref struct

A `ref struct` is a value type that lives only on the stack. It cannot be boxed or used as a field of a class.

```
public ref struct Matrix3x3
{
    public float M11, M12, M13;
    public float M21, M22, M23;
    public float M31, M32, M33;
}
```

BenchmarkDotNet

BenchmarkDotNet is the gold standard for microbenchmarking in .NET:

```
[MemoryDiagnoser]
public class StringBenchmarks
{
    [Benchmark]
    public string Concat() => string.Concat("a", "b", "c");

    [Benchmark]
    public string Interpolation() => $"{'a'}{'b'}{'c'}";
}
```

Extensibility

Managed Extensibility Framework (MEF) provides a plug-in architecture:

```
[Export(typeof(IPlugin))]
public class MyPlugin : IPlugin { }

// Composition
var catalog = new AssemblyCatalog(Assembly.GetExecutingAssembly());
var container = new CompositionContainer(catalog);
var plugins = container.GetExports<IPlugin>();
```

Custom MSBuild tasks let you extend the build pipeline:

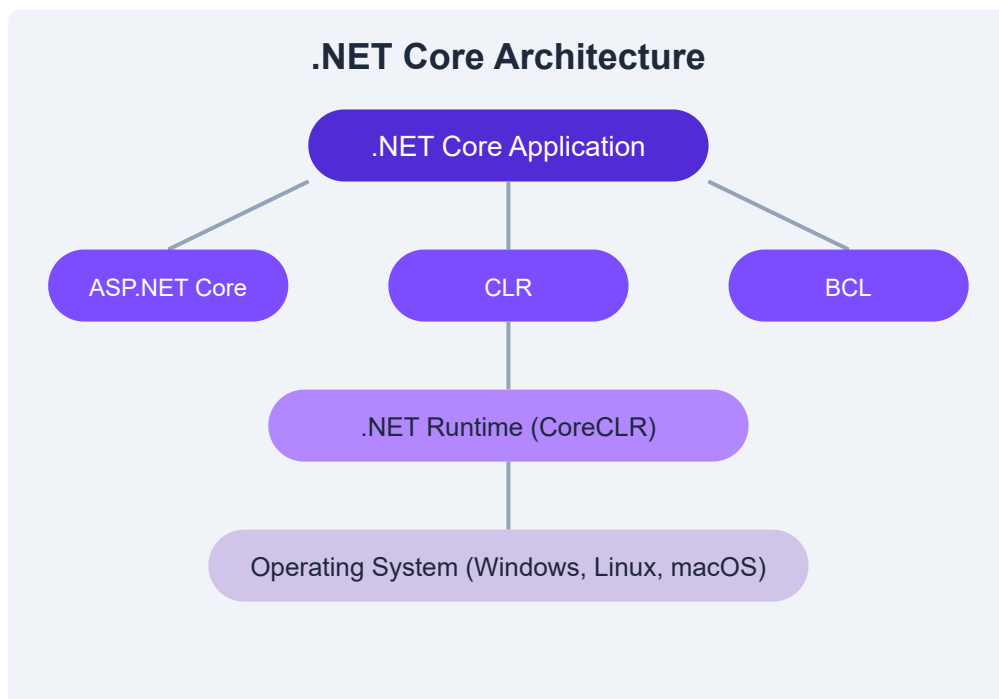
```
public class MyTask : Microsoft.Build.Utilities.Task
{
    public override bool Execute()
    {
        Log.LogMessage("Custom task running...");
        return true;
    }
}
```

Interop

P/Invoke calls native C functions:

```
[DllImport("user32.dll", CharSet = CharSet.Auto)]
public static extern int MessageBox(IntPtr hWnd, string text, string caption,
```

COM Interop allows consuming COM objects. Generate interop assemblies with `tlbimp.exe` or use `dynamic` with `System.Runtime.InteropServices`.



Diagnostics

.NET Core includes powerful diagnostic tools:

```
dotnet tool install -g dotnet-counters
dotnet tool install -g dotnet-dump
dotnet tool install -g dotnet-trace

dotnet-counters monitor --process-id 1234
dotnet-dump collect --process-id 1234
dotnet-trace collect --providers Microsoft-DotNETCore-SampleProfiler
```

Use these tools to monitor CPU, memory, garbage collection, and thread pool metrics in production.

Reflection

Reflection enables inspecting types, invoking members, and creating instances at runtime:

```
Assembly asm = Assembly.LoadFrom("MyLibrary.dll");
Type type = asm.GetType("MyLibrary.MyClass");
object obj = Activator.CreateInstance(type);
MethodInfo method = type.GetMethod("MyMethod");
method.Invoke(obj, new object[] { "arg" });

// Custom attributes
[AttributeUsage(AttributeTargets.Class)]
public class DisplayNameAttribute : Attribute
{
    public string Name { get; }
    public DisplayNameAttribute(string name) => Name = name;
}
```

Source Generators

Source generators produce C# source files during compilation. They enable compile-time code generation without reflection overhead.

```
[Generator]
public class HelloGenerator : ISourceGenerator
{
    public void Execute(GeneratorExecutionContext context)
    {
        var source = "namespace Gen { public class Hello { public static stri
        context.AddSource("Hello.g.cs", source);
    }

    public void Initialize(GeneratorInitializationContext context) { }
}
```

Source generators are commonly used for auto-generating serialization code, dependency injection registrations, and mapper implementations.

Exercise: Write a source generator that scans all classes in the compilation marked with `[AutoLog]` and generates a partial class with an `ILogger` property and a `LogMethodEntry/Exit` helper. Then use BenchmarkDotNet to compare the performance of `string.Concat` vs `StringBuilder` for 100 concatenations.

Practice & Resources

This chapter provides a quick reference, curated learning resources, a study plan, interview preparation, and answers to frequently asked questions.

Quick Reference Guide

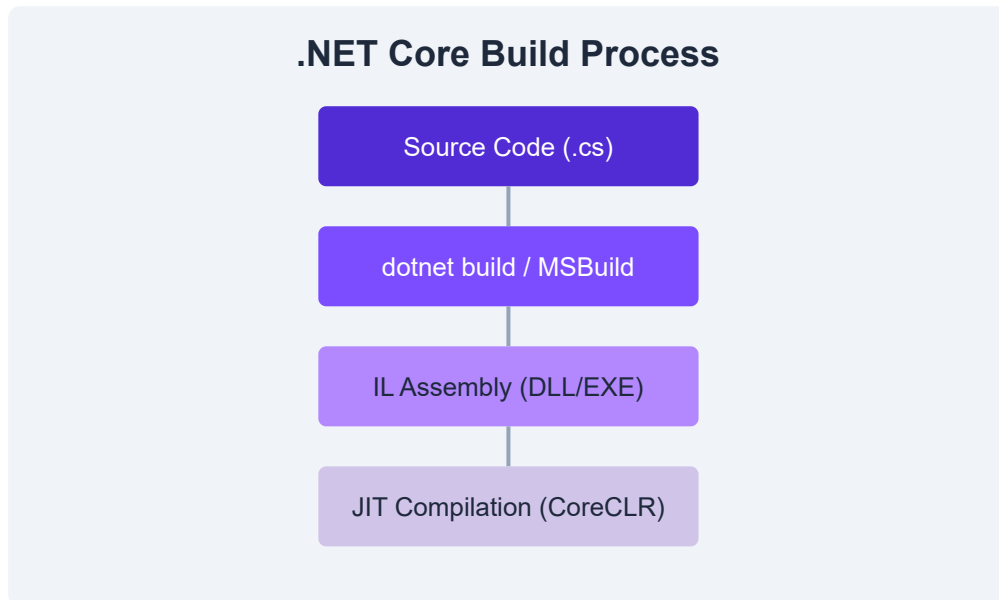
Task	Command
Create console app	<code>dotnet new console</code>
Create web API	<code>dotnet new webapi</code>
Build project	<code>dotnet build</code>
Run tests	<code>dotnet test</code>
Add NuGet package	<code>dotnet add package <name></code>
Publish app	<code>dotnet publish -c Release</code>
Create NuGet package	<code>dotnet pack</code>
Push NuGet package	<code>dotnet nuget push</code>
List SDKs	<code>dotnet --list-sdks</code>

Useful Resources

- **Official Documentation:** docs.microsoft.com/dotnet
- **.NET GitHub Organization:** github.com/dotnet
- **.NET Blog:** devblogs.microsoft.com/dotnet
- **NuGet Gallery:** nuget.org
- **ASP.NET Core Documentation:** docs.microsoft.com/aspnet/core

Discussion Forums

- **Stack Overflow** — Tag your questions with `#.net` , `#.net-core`
- **.NET Community on GitHub Discussions** — [dotnet/core/discussions](https://github.com/dotnet/core/discussions)
- **Reddit** — [/r/dotnet](https://www.reddit.com/r/dotnet) , [/r/csharp](https://www.reddit.com/r/csharp)
- **.NET Foundation Slack** — dotnetfoundation.slack.com



6-Week Study Plan

1. C# syntax, data types, control flow, methods. Complete Chapters 1-2.
2. OOP concepts, collections, LINQ, exception handling. Chapters 3-4.
3. .NET Core architecture, CLI, project structure. Chapters 5-7.
4. Libraries, NuGet, testing, dependency injection. Chapters 8-9.
5. Build, deployment, Docker, CI/CD, migrations. Chapters 10-12.
6. Advanced topics, source generators, performance. Chapters 13-15.

Interview Prep Questions

- Explain the difference between `class` and `struct` in C#.
- What is the Garbage Collector and how does generational collection work?
- Describe the `async` / `await` pattern and how `SynchronizationContext` affects it.

- What is the difference between `IEnumerable<T>`, `ICollection<T>`, and `IList<T>`?
- How does middleware work in ASP.NET Core?
- What are the benefits of dependency injection and the different lifetimes?
- Explain `Span<T>` and when you would use it.

Bootcamp Recommendations

Self-paced bootcamps include Microsoft Learn's **.NET Learning Path**, **Pluralsight .NET Path**, and **Udemy's Complete .NET Core Course**. Many are free with a Microsoft account.

Resume Builder Tips

- Highlight specific .NET versions and frameworks you have used (e.g., ASP.NET Core 8, Entity Framework Core).
- List measurable accomplishments: "Reduced API response time by 40% using `Span<T>` and caching."
- Include open-source contributions to .NET projects on GitHub.
- Mention experience with CI/CD tools (Azure DevOps, GitHub Actions).

Q&A

Q: Is .NET Core free?

A: Yes, .NET Core and .NET 5+ are fully open-source under the MIT license.

Q: Can I run .NET Core on Linux?

A: Yes, Linux is a first-class target for .NET.

Q: Should I use .NET Framework or .NET 8 for new projects?

A: Always choose the latest .NET (8+) for new projects. .NET Framework is legacy.

Glossary

CLR

Common Language Runtime — the virtual machine that executes .NET programs.

BCL

Base Class Library — the standard library of foundational types.

JIT

Just-In-Time compiler that converts IL to native machine code at runtime.

AOT

Ahead-Of-Time compilation (Native AOT) that produces fully native binaries.

IL

Intermediate Language — the CPU-independent bytecode produced by C# compilers.

TFM

Target Framework Moniker, e.g. `net8.0`, `netstandard2.0`.

Exercise: Create a study schedule based on the 6-week plan above. For each week, build a small project (console app, class library, web API, test suite, Docker image). Document your progress and share it on GitHub with a README linking to this tutorial.

References

A condensed reference for .NET Core runtime, libraries, SDK commands, compiler switches, API surface, and NuGet packaging. Bookmark this chapter for quick lookup during development.

.NET Core Runtime Reference

The runtime (`dotnet`) hosts and executes managed applications. Key concepts:

- **dotnet.dll** — The managed entry point that boots the CLR.
- **Host** (`dotnet.exe`) — The native host that loads the runtime.
- **Runtime pack** — Target-specific runtime assets under `Microsoft.NETCore.App` .
- **AppHost** — A native executable specific to your app, configurable via `--apphost` .

```
dotnet --info                # Runtime environment details
dotnet --list-runtimes       # Installed runtimes
dotnet --list-sdks           # Installed SDKs
dotnet --version             # Current SDK version
```

Runtime Configuration

Runtime behavior can be tuned via `runtimeconfig.json` or environment variables:

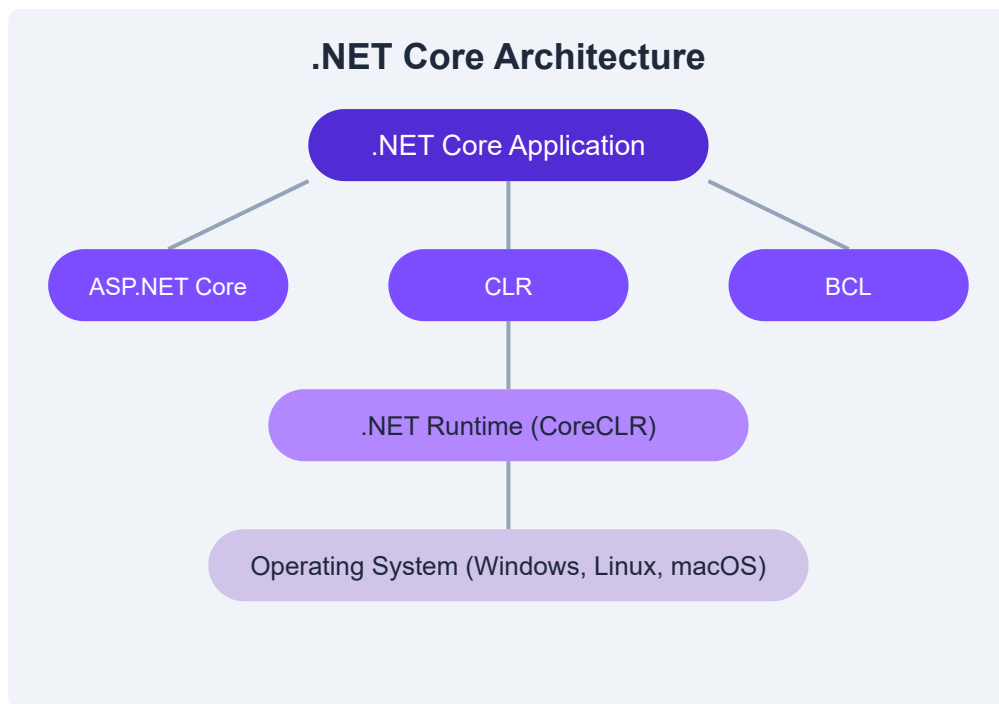
```
{
  "runtimeOptions": {
    "configProperties": {
      "System.GC.Server": true,
      "System.GC.Concurrent": true,
      "System.Threading.ThreadPool.MinThreads": 4,
      "System.Runtime.Serialization.EnableUnsafeBinaryFormatterSerialization"
    }
  }
}
```

Compiler Reference (csc / dotnet build)

The C# compiler (`csc`) is invoked automatically by `dotnet build` . Key switches:

Switch	Description
<code>-target:exe</code>	Build a console executable
<code>-target:library</code>	Build a DLL
<code>-out:<file></code>	Specify output file name
<code>-optimize+</code>	Enable compiler optimizations
<code>-langversion:12</code>	Set C# language version
<code>-nullable:enable</code>	Enable nullable reference types
<code>-define:DEBUG</code>	Define preprocessor symbol
<code>-p:<prop>=<val></code>	Pass MSBuild property (via dotnet build)

```
dotnet build -p:Optimize=true -p:DefineConstants=TRACE
dotnet build -p:WarningLevel=5 -p:TreatWarningsAsErrors=true
```



API Reference

The .NET Base Class Library provides thousands of types. Below are the most essential namespaces:

System

`Object`, `String`, `Int32`, `Boolean`, `DateTime`, `TimeSpan`, `Guid`, `Uri`, `Version`,
`Enum`, `Convert`, `Math`, `Random`, `Console`, `Environment`.

```
string path = Environment.GetFolderPath(Environment.SpecialFolder.MyDocuments);
int hash = "hello".GetHashCode();
Guid id = Guid.NewGuid();
DateTime now = DateTime.UtcNow;
```

System.Collections

`List<T>`, `Dictionary<K,V>`, `HashSet<T>`, `Queue<T>`, `Stack<T>`, `ArrayList`, `Hashtable`,
`IEnumerable<T>`, `ICollection<T>`, `ReadOnlyList<T>`.

```
var dict = new Dictionary<string, int> { ["one"] = 1, ["two"] = 2 };
var list = new List<int> { 1, 2, 3 }.AsReadOnly();
```

System.IO

`File` , `FileInfo` , `Directory` , `DirectoryInfo` , `Path` , `StreamReader` , `StreamWriter` , `FileStream` , `MemoryStream` , `BinaryReader` , `BinaryWriter` .

```
string content = File.ReadAllText("data.txt");
var files = Directory.EnumerateFiles(@"C:\logs", "*.log");
```

System.Linq

`Enumerable` provides extension methods: `Where` , `Select` , `OrderBy` , `GroupBy` , `Aggregate` , `Any` , `All` , `Count` , `First` , `FirstOrDefault` , `Single` , `Skip` , `Take` , `Distinct` .

```
var result = numbers.Where(n => n > 5).Select(n => n * 2).OrderByDescending(n
```

System.Threading.Tasks

`Task` , `Task<T>` , `ValueTask` , `ValueTask<T>` , `Parallel` , `Task.Run` , `Task.WhenAll` , `Task.WhenAny` , `CancellationToken` , `CancellationTokenSource` .

```
var data = await httpClient.GetStringAsync(url);
var results = await Task.WhenAll(task1, task2, task3);
```

For the full API reference, visit docs.microsoft.com/dotnet/api.

NuGet Package Reference

NuGet is the .NET package manager. Key commands and conventions:

```
dotnet add package Newtonsoft.Json --version 13.0.3
dotnet remove package Newtonsoft.Json
dotnet list package

# Package reference in .csproj:
<PackageReference Include="Newtonsoft.Json" Version="13.0.3" />
```

Common package sources: [nuget.org](https://www.nuget.org), GitHub Packages, Azure Artifacts, and local/network feeds.

Package Versioning

NuGet uses semantic versioning (semver): `Major.Minor.Patch[-prerelease]`. Use floating versions with caution:

```
<PackageReference Include="MyLib" Version="2.*" />
<PackageReference Include="MyLib" Version="*-*" />
```

Exercise: Using only the API reference above, write a complete .NET console application that reads all `.txt` files from a directory, counts the frequency of each word across all files, and outputs the top 10 most frequent words. Use `System.IO` for file access, `System.Linq` for grouping/ordering, and `System.Collections.Generic` for the dictionary.